

EduCanvas 代码质量与架构质量审计报告

审计基线：`main@245c4262f30a4b15a109858244ccc8a3513f15f1`
最新代表性变更：PR #284 `feat(voice): add sherpa model gate and reliable audio deletion`
审计日期：2026-08-05

一、先把最难听的话说在前面

EduCanvas 现在最大的问题已经不是“代码写得像学生项目”。

它的问题恰恰相反：

工程体系已经膨胀到中大型平台的复杂度，但产品、数据模型和团队规模还没有完全跟上。

你已经建立了很强的测试、审计、失败关闭、Port/Adapter、事务 Outbox、权限边界和文档纪律。但目前出现了三个危险倾向：

- 1. 控制流统一了，数据事实仍然双轨；
- 2. 边界文档写得很强，代码层的边界却有不少只靠自觉；
- 3. 为了证明系统可信，引入了越来越多计划编号、台账、兼容层和重复投影，开始接近“治理系统比产品本身更复杂”。

这不是烂代码，而是典型的高质量技术债：每一块单独看都合理，组合起来却越来越难改变。

二、综合评分

维度	评分	评价
代码基础质量	7.8 / 10	类型、失败语义、测试和命名总体扎实
架构方向	8.2 / 10	单 Agent Loop、可信事实、Port/Adapter 方向正确
架构实际收敛度	6.5 / 10	控制流收敛明显，入口与数据层仍双轨
测试与 CI	9.0 / 10	是仓库当前最强的一部分
模块边界强制力	6.1 / 10	很多边界只写在注释和 ADR 中
前端可维护性	6.6 / 10	已有设计体系，但工作区编排组件正在变成 God Component
团队协作与审查证据	6.0 / 10	PR 和 CI 完整，独立审查记录不足
生产准备度	5.8 / 10	仓库自己也诚实承认尚未达到生产声明条件

综合评价：7.3 / 10。

它明显强于普通竞赛项目和大多数个人 Agent 项目，但还没有达到“新成员可以安全进入、持续半年迭代而不依赖作者脑内全局模型”的程度。

三、严重问题

1. Agent Loop 已经统一，但系统事实仍然存在两套世界

这是当前最根本的架构问题。

架构文档强调 Gateway、Web General 和 Web Teaching 都进入同一个 `TurnApplicationService`。这部分确实已经成立。但文档同时承认，三条生产 Turn 路径仍有各自的 Adapter、消息持久化和兼容行为；Web 仍然保留 BFF/SSE 兼容层，教学路径仍然写入部分 K12 专用事实。

数据库公共入口进一步暴露了这种双轨状态：

- `DrizzlePlatformTurnRepository` 与 `DrizzleChatRepository`
- `DrizzleAgentModelRunRepository` 与 `DrizzleModelRunRepository`
- `DrizzleAgentToolCallRepository` 与 `DrizzleToolCallRepository`
- `DrizzlePlatformConversationRepository` 与学习会话体系
- `k12-conversation-dual-write`
- `k12-conversation-parity`

这些不是命名问题，而是在说明：

统一 Agent Loop 下面仍然挂着“通用平台事实”和“旧教学事实”两套持久化语义。

为什么危险

现在测试很多，所以双轨暂时可控。但每新增一个能力，都必须回答：

- 写平台消息还是教学消息？
- 哪个仓储拥有终态？
- 是否需要 dual write？
- 哪个事件用于回放？
- 通用入口和教学入口是否都接上？
- parity test 是否也要更新？

这会持续制造“代码没错，但不敢改”的状态。

我的判断

这是**演进债务**，不是**当前事故**。但如果继续一边增加语音、Memory、Credential、渠道和教学能力，一边保留双轨，未来最大的开发成本不会来自模型，而会来自“到底哪一套事实才权威”。

建议

下一阶段必须明确一个终局：

- `platform_*` / `agent_*` 成为唯一通用事实；
- K12 表只保存真正的教学领域事实，例如诊断、掌握度和学习计划；
- 消息、Turn、模型运行、工具调用不再以“教学版”和“通用版”分别存在；
- dual write 只能作为有明确删除日期的迁移工具。

不要再新增第三套仓储来“平滑迁移”。

2. Web 不是薄 BFF，而是第二个后端平台

`@educanvas/web` 直接依赖了至少 13 个内部 workspace：

- Agent Core / Runtime
- DB
- Gateway Core / Runtime
- Model Gateway
- MCP Runtime
- Node Runtime
- Teaching Core / Runtime
- Canvas Protocol
- Asset Processing
- Telemetry

从架构上说，Web 当前同时承担：

- 浏览器 UI；
- SSE 协议；-匿名/注册身份兼容；
- Gateway Operation 创建；
- 通用 Agent 组合根；
- 教学 Agent 组合根；
- Asset materialization；
- Model Runtime 解析；
- Tool Adapter；
- Canvas 服务端授权；
- 一部分持久化生命周期。

这已经不是 BFF，而是与独立 Gateway 并列的第二个服务端平台。

最明显的证据：协议来回翻译

`beginWebGatewayTurn()` 做了以下事情：

1. 将 Web 请求包装成 `GatewayInboundEnvelope` ；
2. 创建 `GatewayService` ；
3. 使用 `WebCompatibilityRunner` 启动 Web 自己的 Turn Application ；
4. 将 Turn Application Event 投影成 Gateway Event ；
5. 最后再通过 `gatewayToLegacy()` 转回旧 Web 事件。

同一文件里还保留了 `legacyToGateway()` 。

这相当于：

```
Web Legacy
→ Gateway Envelope
→ Gateway Service
→ Web Turn Application
→ Gateway Event
→ Web Legacy Event
```

表面上协议统一了，实际上业务仍在 Web 内执行，只是包了一层 Gateway 语义。

直接后果

独立 Gateway 和 Web 的能力不完全一致：

- Gateway General 当前拒绝 Asset ；
- Web General 支持 Asset ；
- Web Teaching 又有独立 Profile、Lifecycle、Tool Adapter ；
- 新能力仍然要在至少两个组合根接线。

Gateway Runner 甚至直接对所有非纯文本输入返回 `CAPABILITY_UNAVAILABLE` 。

建议

不要马上拆更多服务。当前更现实的方案是：

1. 把 Web Server 和 Gateway 的组合根依赖装配抽成一个共享 composition factory ；
2. Web 只保留身份 Cookie、SSE 投影和页面专用 Asset materialization ；
3. 删除双向协议翻译，最多保留单向 canonical event → Web projection ；
4. 为兼容层设退出指标，而不是永久称为“兼容选择”。

3. @educanvas/db 的边界声明与实际 API 完全矛盾

`packages/db/src/index.ts` 开头写的是：

业务层应通过此处导入，避免绕过统一连接生命周期或绑定内部文件结构。

紧接着却公开导出了：

```
export { getDb } from './client';
export * from './schema';
export * from './schema/study';
```

然后又导出了几乎所有表、Repository、错误类型和内部数据结构。

这意味着任何业务代码都可以：

- 直接拿 `getDb()` ；
- 直接 import 表 ；
- 绕过 Repository ；
- 在应用层随手写 Drizzle 查询 ；
- 绕过仓储中已有的租约、所有权、幂等和状态机约束。

最新 PR 的集成测试就合理地利用了这些出口直接操作数据库，但由于生产代码和测试代码使用同一个公共入口，包本身根本无法阻止生产代码照做。

这不是说当前已经大量滥用

我没有证据证明业务层已经普遍绕过 Repository。

问题在于：

所谓数据库边界目前是团队约定，不是代码边界。

建议

不要立刻拆成十几个数据库包。先做最小治理：

@educanvas/db	只导出稳定 Repository / Port 实现
@educanvas/db/schema	仅迁移、DB 内部和受控集成测试使用
@educanvas/db/testing	seed、cleanup、测试数据库工具
@educanvas/db/unsafe	getDb，名称直接表达风险

再用 ESLint `no-restricted-imports` 限制 `apps/**` 直接引用 schema 和 `getDb`。

4. `agent-core` 已经从“Agent Core”变成了平台公共杂物间

`packages/agent-core/src/index.ts` 目前统一导出了：

- Asset contracts
- Message contracts
- Model contracts
- Model Gateway ports
- Model Run ledger
- Turn Application
- Operation continuation
- Tool call / Tool effect
- Object storage
- Web Runtime
- Streaming transcription
- Experiment Runtime

这些模块并非毫无关系，但它们显然不再属于一个高内聚的“Agent Core”。

特别是：

- Web Runtime 不等于 Agent Core ；
- 对象存储不等于 Agent Core ；
- 流式转录协议不等于 Agent Core ；
- Experiment Runtime 也不应天然成为所有 Agent 消费者的公共认知负担。

当前风险

每个应用只需要其中几个类型，却被鼓励统一从根 barrel 导入。久而久之：

- 包所有权模糊；
- 任何新能力都倾向于扔进 `agent-core` ；
- 依赖图看似整洁，语义边界却持续变差；
- 很难判断 breaking change 属于 Agent、媒体还是 Runtime。

不建议

不要立刻把它拆成二十个 workspace。你现在已经有包数量膨胀问题，再机械拆包只会更糟。

建议

先使用 subpath exports：

```
@educanvas/agent-core/turn
@educanvas/agent-core/model
```

```
@educanvas/agent-core/tools
@educanvas/runtime-contracts/web
@educanvas/runtime-contracts/experiment
@educanvas/media-contracts/transcription
```

随后禁止继续向根 `index.ts` 增加新的无关领域。

四、代码质量层面的明确问题

5. 兼容层用展示文案推断协议身份，非常脆弱

`web-turn.ts` 中：

```
function toolName(label?: string): string {
  if (label?.includes('搜索')) return 'web.search';
  if (label?.includes('读取')) return 'web.fetch';
  return 'agent.tool';
}
```

这意味着 canonical tool name 是从中文展示文案猜出来的。

同一兼容投影还会：

- 把所有 `tool.failed` 映射为 `CAPABILITY_UNAVAILABLE`；
- 一律标记 `retryable: true`；
- 通过 `event.code.includes('rate_limit')` 和 `includes('aborted')` 猜错误类型。

这是典型的“为了兼容暂时这么写，最后变成正式协议”。

风险

- 文案改成英文，工具身份变化；
- “读取资料”和“读取网页”可能被错误归为同一工具；
- 权限拒绝、参数错误、供应商失败全被压缩成能力不可用；
- 前端重试策略可能因此错误。

修复方向

事件源必须直接携带：

```
tool: 'web.search'  
failureCode: 'POLICY_DENIED' | 'INVALID_INPUT' | ...  
retryable: boolean
```

展示 label 只能用于 UI，绝不能参与协议判断。

6. 前端工作区组件已经成为 God Component

`GeneralChatWorkspace` 超过 500 行，同时负责：

- 对话 Turn；
- Notebook Source；
- Artifact Generation；
- Studio；
- Canvas；
- HTML Preview；
- Sidebar；
- Online 状态；
- Session Storage 恢复；
- GSAP Flip；
- Composer；
- 多种 Sheet；
- Accessibility announcement。

它已经不是“页面组合组件”，而是一个隐式 UI 状态机。

已经出现的重复

同一个文件中重复渲染或装配了：

- Landing Composer 与普通 Composer；
- Artifact Canvas；
- Source Resource Renderer；
- Artifact 回调；
- Preview 全屏逻辑。

这种复制不是简单的 JSX 重复，而会产生行为漂移：未来修复一个分支时，很容易漏掉另一个分支。

更严重的是错误被吞掉

例如：

```
refreshAssets().catch(() => undefined);
```

以及获取 Artifact 失败时直接：

```
.catch(() => setStudioItems([]));
```

这会把“请求失败”伪装成“没有内容”，与仓库强调的可信失败语义冲突。

建议

至少拆出三层：

```
useGeneralWorkspaceController()  
    负责状态、命令、错误和跨功能协调  
  
GeneralWorkspaceLayout  
    负责 Landing / Conversation / Studio 布局  
  
WorkspaceCanvasSlot  
    统一处理 artifact/source/html/pending/error
```

并将 Landing 和 Conversation 共用的 Composer props 收敛到一个构造函数或组件。

这不是为了追求小文件，而是为了让 UI 状态转换可以单独测试。

7. Web 的模块边界没有静态强制

Web 的 ESLint 配置基本只是 Next.js 默认规则：

```
...coreWebVitals,  
...typescript
```

没有：

- client → server import 限制；
- features 之间的层级限制；
- UI → DB / Runtime 禁止规则；
- 循环依赖检测；
- workspace boundary 规则。

虽然不少服务端文件使用了 `import 'server-only'`，这是好事，但它只能防止部分错误打进客户端包，不能维持整体模块架构。

建议

增加最少量的强制规则：

- `features/**` 禁止直接依赖 `@educanvas/db`；
- Client Component 禁止依赖 `model/gateway/runtime/db`；
- UI 组件禁止反向 import workspace 页面；
- 跨 feature 只能通过各自的 `public.ts` 或显式出口。

8. 注释正在获得超出代码事实的权威

仓库非常重视“AI-readable boundary comments”，甚至专门合并过只增加边界注释的 PR。这种做法有价值，但已经出现了注释与事实不一致。

Worker 模型配置中写道：

主配置解析一次并缓存本轮。

但 `readPrimaryConfiguration()` 并没有缓存，每次都会重新读取环境并调用 `parseModelGatewayConfiguration()`；`resolveMediaConfiguration()` 又会重新读取一次环境。

Web 模型运行时也写道：

每次 Turn 在服务端解析一次配置。

实际代码先显式调用 `parseModelGatewayConfiguration()`，随后 `createTurnModelGatewayFromEnvironment()` 内部再次解析，视觉 Gateway 工厂还会再解析一次。

这不是性能事故，配置解析很便宜。

真正的问题是：

当注释被当成 Agent 的架构事实源时，一句过度概括的注释会比普通坏注释造成更大误导。

建议

注释只描述不可从代码直接看出的“为什么”和不变量。

“解析一次”“唯一入口”“绝不发生”这类可验证断言，必须有对应测试或静态约束，否则降低措辞强度。

9. Node 运行时与类型定义版本不一致

仓库明确固定 CI 和本地主要 Node 版本为 22：

```
.nvmrc = 22
```

但多个 workspace 使用：

```
"@types/node": "^26.1.1"
```

例如 Web、Gateway、Agent Core、DB。

这会让 TypeScript 允许使用 Node 26 才存在、Node 22 尚未支持的 API。

CI 类型检查依然会通过，运行时才可能失败。

修复

如果生产和 CI 基线是 Node 22，就将 `@types/node` 固定到 Node 22 对应 major。

若准备升级 Node 26，则应：

- 修改 `.nvmrc` 和 `engines`；
- 增加 Node 22/26 矩阵；
- 明确最低运行版本；
- 不要处于“运行时 22、类型 26”的中间态。

10. 有些代码复杂度是人为制造的

Gateway 对输入模态的判断：

```
if (  
  parts.some(part => part.type !== 'text' && part.type !== 'asset_ref') ||  
  parts.some(part => part.type === 'asset_ref')  
)
```

逻辑上等价于：

```
if (parts.some(part => part.type !== 'text'))
```

它可能是演进过程中留下的写法，但这类代码在仓库里非常危险，因为读者会误以为两个条件分别承载不同安全语义。

类似问题不算 Bug，却说明项目出现了“为了显得边界严密，让代码表达比业务规则更复杂”的倾向。

五、工程治理问题

11. 计划文档已经成为第二套会漂移的状态数据库

最新 PR #284 明确记录：

- V09 PASS；
- V15 PASS；
- 下一项为 V12。

UV-画布语音.md 顶部也已经写成下一任务 V12。

但 docs/plan/active/README.md 仍然写着：

- V09/V15“已解锁”；
- 下一步可并行执行 V09、V15。

架构现状文档最后验证时间仍为 2026-07-25，并描述 U13、对象删除等当时尚未完成的状态，而主干已经推进到 U16、V15 之后。

这暴露了什么

当前项目同时存在多个“事实源”：

- 代码；
- 测试；
- PR body；
- Active README；
- 各任务线文档；
- ADR；
- 验证台账。

当这些都需要人工同步时，漂移是必然的。

建议

只保留一个机器可解析的状态源，例如：

```
tasks:
  V09:
    status: pass
    evidence:
      - pr: 284
      - commit: 245c4262
  V12:
    status: unlocked
```

README、任务表和阶段概览由脚本生成或验证。

文档仍可详细解释，但不再手写重复状态。

12. CI 很强，独立审查证据却很弱

最新 PR #284 的 CI 是成功的，实际覆盖：

- Secret Scan ；
- Lint ；
- Typecheck ；
- Unit ；
- PostgreSQL Integration ；
- Windows Smoke ；
- Runtime Pressure ；
- Production Build ；
- Persistent Runtime Composition ；
- Browser E2E。

这是非常值得肯定的工程基础。CI 工作流本身也清晰地把这些任务设成独立 Job。

但 PR #284 ：

- 没有 requested reviewer ；
- 没有 review ；
- 没有讨论评论。

对于个人项目，这不是什么道德问题。但对仓库里反复出现的“Codex 审核通过”“独立 Codex review”来说，GitHub 上缺乏可追责的审查记录。

风险

目前的治理模型依然是：

作者写代码

- 作者或同一工作流运行 Agent 审核
- 作者记录 PASS
- 作者合并

这更接近高质量自检，而不是独立审查。

建议

至少对高风险变更要求：

- PR 上留下正式 review ；
- review 明确基于哪个 commit SHA ；
- 列出阻塞项与非阻塞项 ；
- 修复后重新确认 ；
- 数据迁移、安全边界、权限或删除承诺必须有 `APPROVE` 记录。

六、值得保留的强项

这轮审计不是为了无脑贬低。以下部分确实做得很好。

1. CI 不是装饰品

很多项目 README 写了测试，实际只跑 lint。EduCanvas 当前真实执行了完整的单元、集成、浏览器、运行时压力、Windows 和 Secret Scan 门禁。

2. Agent 主循环拆分质量明显提升

`TurnApplicationService` 已经不再是一整个巨型文件，而是拆成：

- preparation ；
- loop runner ；
- completion ；
- session ；
- dependencies ；
- ports ；
- helpers。

主服务清楚维护 replay、取消、审批、终态和失败收敛。

它仍然复杂，但复杂度大体来自真实业务，而不是单纯堆 if/else。

3. 失败关闭纪律是真实存在的

Gateway 不支持的模式明确失败，不会伪装接通；Model Gateway 未配置时返回不可用；对象删除使用租约、attempt 和 Outbox；集成测试拒绝连接非测试数据库。

这些不是 README 口号，在实现与测试中都有体现。

4. 安全边界意识高于多数 Agent 项目

当前抽查范围内，没有发现：

- Provider Key 暴露给浏览器；
- Provider 原始类型穿透公共契约；
- 主页面直接执行模型生成代码；
- 删除失败后静默声称物理删除完成；
- 集成测试误连生产库的明显路径；
- 模型直接写掌握度或课程事实。

这不等于完成安全审计，但说明基础方向是正确的。

5. 项目会诚实记录失败实验

语音模型评估没有为了推进路线而篡改指标，而是明确记录 WER、术语召回和风险接受。这种证据纪律是项目最有价值的部分之一。

七、整改优先级

P0：当前没有发现必须立刻停服的致命问题

我没有在本轮抽查中发现明确的：

- 跨租户越权；
- 密钥泄露；
- 任意代码执行逃逸；
- 数据库破坏；
- 删除承诺直接失效。

但这不是完整安全审计结论。

P1：下一阶段必须处理

A. 冻结并收敛双轨持久化

验收标准：

- 明确平台消息、Turn、Model Run、Tool Call 的唯一事实表；
- 列出旧 K12 仓储的调用者；
- dual write 有删除里程碑；
- 新能力不得再写旧仓储。

B. 收口 Web 与 Gateway 双组合根

验收标准：

- 通用 Turn 依赖装配只有一个 canonical factory；
- Web 不再进行 Gateway → Legacy → Gateway 式双向翻译；
- canonical tool id 和 failure code 不从展示文案推断；
- Asset/K12 能力差异有显式 capability matrix。

C. 强制数据库边界

验收标准：

- 默认 `@educanvas/db` 不再导出 `getDb` 和 `schema`；
- 应用代码直接 `schema import` 在 lint 中失败；
- 集成测试通过专门 `testing export` 获取底层访问。

P2：随后处理

D. 拆解 `GeneralChatWorkspace`

目标不是把一个文件机械拆成五个文件，而是形成可测试的 UI 状态模型。

E. 为 `agent-core` 建立 subpath 和所有权

先停止继续向根 barrel 倾倒协议，再逐步迁移。

F. 建立单一计划状态源

至少让 CI 能发现：

- Active README 与任务文件状态矛盾；
- PASS 任务没有 PR/commit evidence；
- 已归档任务仍出现在 active 表中。

G. 对齐 Node Runtime 与类型版本

这是成本极低、收益明确的修复。

八、最终定性

EduCanvas 当前不是“架构差”。

它是：

架构思想很强、局部实现也很扎实，但正在被兼容层、双轨事实、公共 barrel、文档台账和单人治理复杂度反噬。

最值得警惕的不是某个函数写得丑，而是下面这个趋势：

```
每增加一个能力
→ 增加一个协议
→ 增加一个 Adapter
→ 增加一个状态台账
→ 增加一组兼容投影
→ 增加几十个测试
→ 但旧路径没有真正删除
```

如果继续这样推进，EduCanvas 会成为一个“几乎什么都有、几乎什么都经过验证，但没人敢大改”的系统。

下一阶段不应该继续证明你会做更多工程设施。

下一阶段应该证明：

你有能力删除旧架构、减少概念数量，并让一个新开发者在不阅读二十份 ADR 和任务台账的情况下，仍能安全完成一项真实功能。